

Project title:

Employee management system

Team members:

Student name	Id number
Fatma maged mohammed	2023/05656
Shahd waleed samy	2024/05549
Sama Mohamed Mohamed Abd El moniem	2024/00293
Salma Mahmoud Seliem	2024/08613

1. Overview of the System Design and Architecture

A short paragraph summarizing the purpose and features of the project.

An Employee Management System is a comprehensive tool that automates and simplifies various human resource tasks, enabling organizations to manage their workforce more effectively. It acts as a centralized hub for maintaining employee records, monitoring attendance and leave, handling payroll, evaluating performance, and facilitating recruitment and onboarding processes. With built-in compliance and reporting features, the system helps organizations meet legal requirements and generate accurate reports. Common functionalities include managing employee information, tracking work hours, processing salaries, storing documents, and providing secure, role-based access. Overall, an EMS boosts efficiency, reduces manual effort, and enhances organizational performance.

1.1 Modules Overview:

Explain each module used in the project.

The Main Module serves as the program's core, controlling execution flow and user interaction through a menu-driven interface. The Employee Data Module defines the structure for employee records, including enums for departments and job titles, and provides functions to validate and retrieve department and job title names. The Employee Management Module handles all employee-related operations such as adding, displaying, searching, updating, promoting, removing, and moving employees between departments. The Salary Calculation Module computes employee salaries based on hours worked and pay rate, ensuring accurate financial reporting. The File I/O Module manages data persistence by saving employee records to and loading them, handling file operations securely. Additionally, the Input/Output Utilities Module ensures proper input validation, output formatting, and user prompts, while the Validation Module enforces data integrity by checking ID uniqueness, department validity, and job title compatibility. Together, these modules create a structured, maintainable, and efficient system for managing employee data.

example:

- Main Module: Controls program execution and user interaction.
- File I/O Module: Saves and loads data.
-etc.

1.2 Data Structures Used:

-Describe the data structures used in the project like 1D arrays, 2D arrays, structs,..etc.

the system employs a struct named Employee that encapsulates all relevant employee attributes, including ID, name, hours worked, hourly salary, job title, and department. Employee records are stored in a static one-dimensional array (employees[MAX_EMPLOYEES]), allowing for straightforward indexing and iteration during operations like adding, searching, or updating records. The system uses enumerations (Departments and JobTitles) to define valid department categories (HR, Sales, Customer Service, Finance) and corresponding job positions, ensuring data

consistency and preventing invalid assignments. String objects (std::string) handle textual data such as employee names.

2. Description of Each Function and Its Purpose

Function name	Purpose
displayMainMenu()	Displays the 12-option menu
addEmployee()	Adds new employee with validation
displayAllEmployees()	Lists all employees
displayDepartmentEmployees()	Shows employees filtered by department
searchEmployee()	Finds employees by name/ID + department
updateEmployee()	Edits employee details
promoteEmployee()	Upgrades job title + salary
removeEmployee()	Deletes employee record
moveEmployee()	Transfers employee to new department
calculateSalary()	Computes salary (hours × rate)
saveToFile()	Saves all data to file
loadFromFile()	Loads data from file
clearInput()	Clears invalid input buffer
isIdUnique()	Checks for duplicate employee ID
isValidDepartment()	Validates department number (1-4)

<code>isValidJobTitle()</code>	Checks job title matches department
<code>getDepartmentName()</code>	Converts department code to name
<code>getJobTitleName()</code>	Converts job title code to name
<code>displayEmployeeDetails()</code>	Shows formatted employee record

3. Instructions for Compiling and Running the Program

Describe the instructions needed to run the program or any system requirements.

To run the Employee Management System program, you'll need a C++ compiler (Microsoft Visual C++) installed on your Windows, macOS, or Linux system. First, save the provided code into a file named "employee_management.cpp". Then, compile the program using the command "g++ employee_management.cpp -o employee_management" (adding ".exe" on Windows). Once compiled, execute the program by running "./employee_management" (or "employee_management.exe" on Windows). The system will launch with an interactive menu offering options to add, remove, search, and update employee records, calculate salaries, and manage data persistence. The program automatically handles data storage in a text file named "employees.txt", creating it on first run if it doesn't exist. No additional libraries or dependencies are required beyond a standard C++ development environment. The system includes built-in input validation to ensure data integrity and will work seamlessly on any platform with a proper C++ compiler setup. For troubleshooting, verify your compiler installation and ensure you have proper file read/write permissions in the program's working directory.

4. Sample Input/Output Demonstration

```
Enter your choice: 4
Enter employee name: fatma
```

```
Departments:
1. Human Resources
2. Sales
3. Customer Service
4. Finance
Enter department number: 4
```

```
Employee found:
```

```
-----
ID: 5656
Name: fatma
Department: Finance
Job Title: Manager
Hours Worked: 15.00
Salary per Hour: 150000.00
Monthly Salary: 2250000.00
-----
```

```
Salary calculation for salma:
Hours worked: 16
Salary per hour: 910000
Total salary: 14560000.00
```

Employees in Sales department:

ID	Name	Job Title	Hours	Salary/Hour
5549	shahd	Administrator	12	10000
293	sama	Administrator	13	16000

```
5549,shahd,12,10000,3,2
5656,fatma,15,150000,1,4
293,sama,13,16000,3,2
8613,salma,16,910000,9,3
```

5. flowcharts

